ATIVIDADE PRÁTICA 2 (AV2) · SISTEMAS OPERACIONAIS 2026.2

# GenResearch-Servidor

## Memória, armazenamento, concorrência e desempenho sob cargas de IA generativa

Estudo prático de chamadas de sistema, mmap, paginação, exclusão mútua e filas em Linux

---


EQUIPE · 7 INTEGRANTES

Bruno Saint Clair Silva Oliveira
João Victor Oliveira Moura
Leonardo de Souza Aragão
Yan Victor Araujo do Nascimento
Gabriel Bastos Pimentel
José Cerqueira da Silva Neto
Marcus Antônio Martins Oliveira Júnior

CENÁRIO E PERGUNTA NORTEADORA

# “Como memória principal, memória virtual, armazenamento, caches, filas compartilhadas e sincronização afetam o desempenho, a estabilidade e a correção de uma aplicação de IA generativa ou RAG?”

### O que medimos

- Custos de SO: locks, page faults, context switch
- Page cache e subsistema de memória virtual
- Contenção em seções críticas concorrentes

### Fora do escopo

- Acurácia ou qualidade textual da LLM
- APIs em nuvem ou modelos remotos
- Qualquer coisa além do runtime local

AMBIENTE EXPERIMENTAL

# Inventário do ambiente de execução

## Hardware e kernel

| Item | Configuração |
|---|---|
| CPU | Intel Core i5 · 12 threads lógicas |
| RAM | 24 GB DDR5 (23.731 MB visíveis) |
| Disco | SSD NVMe · ext4 |
| SO | Linux x86-64 · Ubuntu LTS · [uname -r] |
| Kernel | CONFIG_TASK_IO_ACCOUNTING ativo |

## IA local (Ollama)

| Item | Configuração |
|---|---|
| GPU | RTX 3050 · 6 GB GDDR6 |
| Embeddings | nomic-embed-text · 768 dim. |
| Geração | LFM2.5-2.6B · Q4_0 (~1,6 GB) |
| Runtime | Python 3.12 · FastAPI · Uvicorn |
| Dados | NumPy · SQLite |

ARQUITETURA DE SOFTWARE

# Arquitetura do GenResearch-Servidor




> O corpus fica no disco: só os **top-k trechos** relevantes são trazidos do SQLite para a RAM a cada consulta.

PIPELINE RAG

# Ingestão e recuperação semântica

## INGESTÃO

**Chunking**
trechos configuráveis com overlap

→

**Embeddings**
nomic-embed-text, 768 dimensões, L2

→

**Persistência**
trechos no SQLite, matriz em vectors.npy

## CONSULTA E GERAÇÃO

**Vetorizar**
query vira vetor

→

**Similaridade**
`matrix @ q`

→

**Top-k**
trechos lidos do SQLite

→

**Contexto**
limite estrito de caracteres

→

**Geração**
LFM2.5 local na GPU

> Cada etapa é cronometrada separadamente: chunking, embeddings, similaridade e geração.

# Fila compartilhada: produtores e consumidores

PRODUTORES (HTTP)

POST /tasks/query

→ **202 Accepted**




CONSUMIDORES

**Pool de workers**

`not_empty.wait()`

### Sem busy-waiting

Fila vazia: o worker dorme na variável de condição e não gasta CPU

### 1, 2 ou 4 workers

Pool redimensionado em tempo de execução

### Telemetria

Ocupação e latência da fila expostas pela API

CACHE E SATURAÇÃO

# Cache LRU compartilhado e backpressure

## Cache LRU em RAM

```
chave = SHA-256(pergunta normalizada + top_k +
max_context_chars)
```

- OrderedDict ordenado por recência de acesso
- Cheio: descarta o item menos recentemente usado
- Toda leitura e escrita sob threading.Lock

## Backpressure na fila

1. Fila cheia: a inserção bloqueia
2. Espera até o timeout de 3 segundos
3. Ainda cheia: **HTTP 503 Service Unavailable**

Impede acúmulo descontrolado de requisições e o esgotamento de memória.

# Experimento D: condição de corrida (lost update)

## Como uma atualização se perde

| t | Thread A | Thread B | contador |
|---|---|---|---|
| 1 | lê 41 | | 41 |
| 2 | | lê 41 | 41 |
| 3 | grava 42 | | 42 |
| 4 | | grava 42 | 42 ✗ |

Dois incrementos, um efeito: o ciclo *read-modify-write* não é atômico.

### Parâmetros

**4** threads, largada via threading.Barrier

**2.000** incrementos por thread

**8.000** valor final esperado

Sem lock vs. com threading.Lock · 3 repetições cada

RESULTADOS DO EXPERIMENTO D · MÉDIA DE 3 REPETIÇÕES

# Corrida versus mutex lock

SEM LOCK · FALHA

**≈2.000**

de 8.000 esperados

**≈75% das atualizações perdidas**

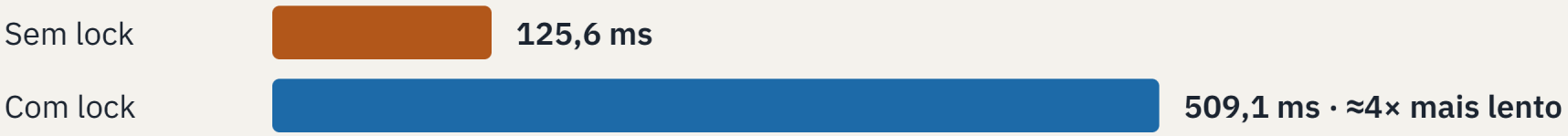
COM threading.Lock · CORRETO

**8.000**

em 100% das repetições

**0 atualizações perdidas**

**Custo da exclusão mútua (duração média)**



# C1 e C2: leitura convencional versus mmap

**C1 · open() + read()**

## 3.500 MB/s

2,79 ms por leitura

Blocos de 64 KB copiados do espaço do kernel para o buffer do usuário.

**C2 · mmap()**

## 2.257 MB/s

4,40 ms por leitura

Arquivo projetado na tabela de páginas, sem cópia, paginado sob demanda pela MMU.

> Arquivo de 9,75 MB já no page cache: o custo das faltas de página do mmap superou o da cópia do read().

TELEMETRIA DO KERNEL · /PROC/SELF/IO E GETRUSAGE

# Page cache e faltas de página

| Por leitura | C1 read() | C2 mmap |
|---|---|---|
| rchar | 10.221.278 B | 158 B |
| read_bytes | 0 B | 0 B |
| Minor faults | 0 | 156 |
| Major faults | 0 | 0 |

**read_bytes = 0:** nenhum byte veio do disco; tudo foi servido pelo page cache em RAM.

**POR QUE 156 FAULTS?**

9,75 MiB ÷ 64 KiB ≈ 156. Cada falta mapeia um bloco de 16 páginas já em cache (*fault-around*), por isso são todas *minor*.

**MEMFREE ≠ MEMÓRIA LIVRE**

MemFree baixo é normal: o kernel usa RAM ociosa como page cache e a devolve quando preciso. O indicador real é MemAvailable.

CONCORRÊNCIA E CACHE · MÉDIAS DE 3 REPETIÇÕES

# Escalabilidade da fila (C3) e eficiência do cache (C4)

## C3 · 20 tarefas, fila de capacidade 10

| Workers | Vazão | Espera | Speedup |
|---|---|---|---|
| 1 | 58 t/s | 115,8 ms | 1,00× |
| 2 | 118 t/s | 56,1 ms | 2,03× |
| 4 | 200 t/s | 26,1 ms | 3,42× |

Escala quase linear até 2 workers; com 4, a contenção na fila já aparece (3,42× de 4×).

## C4 · cache LRU, 10 consultas

Cache frio

**15,15 ms** → **0,01 ms**

Cache aquecido

**100% de acertos** e 99,87% menos tempo por consulta: nenhum acesso redundante ao pipeline.

PRESSÃO DE MEMÓRIA E TOLERÂNCIA A FALHAS

# Paginação sob demanda (C5) e resiliência (C6)

## C5 · um toque a cada 4 KiB

| Alocação | Minor faults | Teórico | RSS pico |
|---|---|---|---|
| 100 MB | 25.602 | 25.600 | 246 MB |
| 250 MB | 64.002 | 64.000 | 396 MB |
| 500 MB | 128.002 | 128.000 | 646 MB |

Uma falta por página de 4 KiB. Após o GC, RSS volta a 146 MB; 0 major faults, sem swap, sem OOM Killer.

## C6 · falhas injetadas, 2 workers

**Transitórias**
10 de 10 recuperadas com até 2 retentativas

**Persistentes**
10 descartadas sem travar o pool

**Saturação da fila**
excedentes rejeitados com HTTP 503

**Workers vivos: 2 de 2** e fila consistente nos três cenários.

RELAÇÃO COM SO

# Sistemas operacionais e IA generativa

C1 · C2

### Page cache

Acelera, de forma transparente, a leitura de índices vetoriais e pesos de modelos.

D · C4

### Locks

Caches e índices compartilhados corrompem sem exclusão mútua; o lock custa latência.

C5

### Paginação e RSS

Contextos e pesos ocupam RAM sob demanda; RSS acima do físico aciona o OOM Killer.

C3 · C6

### Filas e backpressure

Dimensionam a espera e protegem a memória quando a carga excede a capacidade.

LIMITAÇÕES TÉCNICAS

# Limitações e escopo de generalização

**01**

### GIL do Python

Threads não paralelizam trabalho puramente de CPU. Mitigado aqui porque o pipeline é dominado por I/O e espera pela GPU.

**02**

### Métricas por processo

getrusage soma as faltas de página do processo inteiro. Isolar por thread exigiria eBPF com privilégios de root.

**03**

### Nó único

Uma máquina com uma GPU. Clusters com várias GPUs somam custos de barramento, rede e coerência de cache.

CONCLUSÃO

# A estabilidade e o desempenho de IA generativa dependem do dimensionamento de filas, da contenção de locks e do comportamento da memória virtual.

### Artefatos entregues

Relatório técnico, código executável, CSVs de telemetria e gráficos

### Uso transparente de IA

Ferramentas de IA apoiaram texto, formatação e análise preliminar de scripts. Todos os dados, logs e tabelas foram coletados em Linux real, sem números simulados.

# Obrigado!

GENRESEARCH-SERVIDOR · AV2 SISTEMAS OPERACIONAIS